Bab 12

Implementasi Aplikasi

Mengubah rancangan menjadi kode yang berjalan.

12

DI BAB INI

- Implementasi class
- Encapsulation
- Inheritance/composition sesuai rancangan
- Polymorphism bila diperlukan
- Module/package
- File handling
- Integrasi antar-class

Pada Bab 11 kita telah merancang Sistem Peminjaman Perpustakaan: siapa penggunanya, data apa yang diperlukan, dan aturan apa yang harus dijaga. Bab ini mengubah rancangan tersebut menjadi aplikasi Python yang dapat dijalankan melalui terminal. Model dan layanan yang dibuat di sini akan digunakan kembali untuk GUI pada Bab 13.

Kita menggunakan kasus yang sama: satu kode buku mewakili satu eksemplar buku cetak atau satu lisensi e-book. Buku cetak dipinjam selama 14 hari, sedangkan e-book selama 7 hari. Satu buku hanya boleh mempunyai satu peminjaman aktif. Riwayat yang sudah selesai tetap disimpan. Aplikasi digunakan oleh satu petugas pada satu komputer; login, denda, dan akses serentak belum termasuk ruang lingkupnya.

Penerapan class dalam proyek utuh membantu melihat hubungan antara data, aturan, dan pekerjaan pengguna, sebagaimana dibahas dalam proyek terpadu oleh [North, 2025]. Kode dalam bab ini disusun untuk rancangan perpustakaan pada Bab 11, dengan pemisahan komponen agar setiap perilaku dapat diperiksa sebelum dihubungkan ke tampilan [Wilson, 2024].

CAPAIAN PEMBELAJARAN

Setelah mempelajari bab ini, Anda diharapkan mampu:

- menerjemahkan rancangan class dan kontrak method menjadi kode Python;
- menjaga identitas, nilai attribute, dan perubahan status melalui encapsulation;
- menerapkan inheritance dan hubungan antar-object sesuai tanggung jawabnya;
- menggunakan polymorphism untuk variasi perilaku yang memang diperlukan;
- menyusun module dalam package dan menyimpan data beserta relasinya ke JSON;
- mengintegrasikan pendaftaran, peminjaman, pengembalian, serta simpan–muat;
- memeriksa hasil normal dan penolakan transaksi tanpa membuka GUI.

CARA MENGIKUTI KODE DALAM BAB INI

Gunakan Python 3.10 atau yang lebih baru. Seluruh contoh hanya memakai pustaka standar Python. Buat folder proyek baru agar tidak bercampur dengan contoh perpustakaan dari bab sebelumnya. Setiap listing yang berjudul nama file merupakan **isi lengkap file tersebut**; setiap

file hanya perlu dibuat sekali. Listing berjudul `demo_...py` adalah pemeriksaan tambahan, bukan bagian yang digabungkan ke `main.py`.
Lengkapi semua file inti sampai Subbab Integrasi Antar-Class sebelum menjalankan contoh yang mengimpor package `perpustakaan`. Urutan pembahasan mengikuti konsep; beberapa module yang diimpor baru dituliskan pada subbab berikutnya.

12.1 Implementasi Class

Menerjemahkan Tanggung Jawab ke Kode

Class tidak cukup hanya mempunyai attribute yang sama dengan diagram. Methodnya harus memenuhi kontrak: menerima masukan yang sesuai, menjaga aturan, lalu menghasilkan nilai atau exception yang dapat ditangani pemanggil. Pembagian pekerjaan berikut mempertahankan rancangan pada Bab 11.

Tabel 12.1. Tanggung jawab komponen pada implementasi.

Komponen	Tanggung jawab
`Buku`, turunannya	Menyimpan kode dan judul, menyediakan lama pinjam sesuai jenis buku.
`Anggota`	Menyimpan ID anggota dan nama yang valid.
`Peminjaman`	Menghubungkan anggota dengan buku, menyimpan tanggal, dan menyelesaikan satu transaksi.
`Perpustakaan`	Mendaftarkan object, mencari data, memeriksa ketersediaan, dan mengoordinasikan transaksi serta simpan–muat.
`PenyimpananJSON`	Membaca dan menulis dictionary melalui file JSON.
`main.py`	Menerima input terminal dan menampilkan hasil layanan.

Kode buku dan ID anggota merupakan identitas tetap. Judul dan nama boleh diperbarui, tetapi tidak boleh kosong. Object `Peminjaman` menyimpan rujukan ke object anggota dan buku yang sudah terdaftar. Saat data disimpan, rujukan tersebut diubah menjadi ID; saat dimuat, hubungan object dibangun kembali.

Menyiapkan Exception Aplikasi

Buat folder `perpustakaan`, lalu file `kesalahan.py` di dalamnya. Semua exception khusus mewarisi `PerpustakaanError`. Dengan demikian, antarmuka dapat menangkap kesalahan aplikasi melalui satu induk, sedangkan pemeriksaan yang lebih khusus tetap dapat membedakan penyebabnya.

perpustakaan/kesalahan.py

```python
class PerpustakaanError(Exception):
    """Induk kesalahan yang dapat ditampilkan oleh antarmuka."""


class DataTidakValidError(PerpustakaanError):
    pass


class DataTidakDitemukanError(PerpustakaanError):
    pass


class BukuSedangDipinjamError(PerpustakaanError):
    pass


class PeminjamanSelesaiError(PerpustakaanError):
    pass


class DataRusakError(PerpustakaanError):
    pass


class PenyimpananError(PerpustakaanError):
    pass
```

`DataTidakValidError` dipakai untuk masukan seperti nama kosong atau tanggal kembali yang terlalu awal. `DataTidakDitemukanError` berarti ID yang diminta belum terdaftar. `BukuSedangDipinjamError` dan `PeminjamanSelesaiError` menyatakan penolakan transaksi. `DataRusakError` menyatakan isi simpanan tidak dapat dipakai, sedangkan `PenyimpananError` menyatakan kegagalan membuka atau menulis file.

Membuat Model Buku, Anggota, dan Peminjaman

Buat file `model.py` dalam folder yang sama. Listing berikut memuat seluruh class domain, termasuk property dan turunan buku yang akan dibahas lebih rinci pada subbab berikutnya. Model tidak memanggil `input()`, `print()`, atau komponen GUI; hasilnya dapat dipakai oleh antarmuka yang berbeda.

perpustakaan/model.py

```python
from abc import ABC, abstractmethod
from datetime import date, datetime, timedelta

from .kesalahan import DataTidakValidError, PeminjamanSelesaiError
```

```
5
6
7  def teks_wajib(nilai, label):
8      if not isinstance(nilai, str) or not nilai.strip():
9          raise DataTidakValidError(f"{label} harus berupa teks yang terisi.")
10     return nilai.strip()
11
12
13 def tanggal_valid(nilai):
14     if not isinstance(nilai, date) or isinstance(nilai, datetime):
15         raise DataTidakValidError("Tanggal harus berupa object date.")
16     return nilai
17
18
19 class Buku(ABC):
20     JENIS = ""
21
22     def __init__(self, kode, judul):
23         self._kode = teks_wajib(kode, "Kode buku")
24         self.judul = judul
25
26     @property
27     def kode(self):
28         return self._kode
29
30     @property
31     def judul(self):
32         return self._judul
33
34     @judul.setter
35     def judul(self, nilai):
36         self._judul = teks_wajib(nilai, "Judul buku")
37
38     @abstractmethod
39     def lama_pinjam(self):
40         """Mengembalikan lama pinjam dalam hari."""
41
42     def ke_data(self):
43         return {"kode": self.kode, "judul": self.judul, "jenis": self.JENIS}
44
45     def __str__(self):
46         return f"[{self.kode}] {self.judul}"
47
48
49 class BukuCetak(Buku):
50     JENIS = "cetak"
```

```
51
52      def lama_pinjam(self):
53          return 14
54
55
56  class EBook(Buku):
57      JENIS = "ebook"
58
59      def lama_pinjam(self):
60          return 7
61
62
63  class Anggota:
64      def __init__(self, id_anggota, nama):
65          self._id_anggota = teks_wajib(id_anggota, "ID anggota")
66          self.nama = nama
67
68      @property
69      def id_anggota(self):
70          return self._id_anggota
71
72      @property
73      def nama(self):
74          return self._nama
75
76      @nama.setter
77      def nama(self, nilai):
78          self._nama = teks_wajib(nilai, "Nama anggota")
79
80      def info(self):
81          return f"{self.id_anggota} - {self.nama}"
82
83      def ke_data(self):
84          return {"id_anggota": self.id_anggota, "nama": self.nama}
85
86
87  class Peminjaman:
88      def __init__(self, id_pinjam, anggota, buku, tanggal, jatuh_tempo=None):
89          if not isinstance(anggota, Anggota) or not isinstance(buku, Buku):
90              raise DataTidakValidError("Peminjaman memerlukan anggota dan buku.")
91          self._id_pinjam = teks_wajib(id_pinjam, "ID peminjaman")
92          self._anggota = anggota
93          self._buku = buku
94          self._tanggal_pinjam = tanggal_valid(tanggal)
95          if jatuh_tempo is None:
```

```
            try:
                jatuh_tempo = tanggal + timedelta(days=buku.lama_pinjam())
            except OverflowError as exc:
                raise DataTidakValidError("Tanggal di luar jangkauan.") from
    exc
        self._tanggal_jatuh_tempo = tanggal_valid(jatuh_tempo)
        if jatuh_tempo < tanggal:
            raise DataTidakValidError("Jatuh tempo mendahului tanggal pinjam.")
        self._tanggal_kembali = None

    @property
    def id_pinjam(self):
        return self._id_pinjam

    @property
    def anggota(self):
        return self._anggota

    @property
    def buku(self):
        return self._buku

    @property
    def tanggal_pinjam(self):
        return self._tanggal_pinjam

    @property
    def tanggal_jatuh_tempo(self):
        return self._tanggal_jatuh_tempo

    @property
    def tanggal_kembali(self):
        return self._tanggal_kembali

    @property
    def aktif(self):
        return self.tanggal_kembali is None

    def kembalikan(self, tanggal):
        if not self.aktif:
            raise PeminjamanSelesaiError("Peminjaman sudah selesai.")
        tanggal = tanggal_valid(tanggal)
        if tanggal < self.tanggal_pinjam:
            raise DataTidakValidError("Pengembalian mendahului tanggal pinjam.")

        self._tanggal_kembali = tanggal
```

```
140
141     def ke_data(self):
142         kembali = self.tanggal_kembali
143         return {
144             "id_pinjam": self.id_pinjam,
145             "id_anggota": self.anggota.id_anggota,
146             "kode_buku": self.buku.kode,
147             "tanggal_pinjam": self.tanggal_pinjam.isoformat(),
148             "tanggal_jatuh_tempo": self.tanggal_jatuh_tempo.isoformat(),
149             "tanggal_kembali": kembali.isoformat() if kembali else None,
150         }
```

Fungsi `teks_wajib()` memeriksa tipe dan isi teks, kemudian membuang spasi di awal serta akhir. Fungsi `tanggal_valid()` memastikan nilai berupa `date`, bukan teks atau object `datetime`. Pemeriksaan ini menjaga agar operasi tanggal di model mempunyai arti yang sama. Input dari terminal akan dikonversi dengan `date.fromisoformat()` sebelum diteruskan ke layanan.

Parameter `jatuh_tempo=None` pada constructor `Peminjaman` mempunyai dua kegunaan. Untuk transaksi baru, nilainya dihitung dari tanggal pinjam ditambah hasil `buku.lama_pinjam()`. Saat memuat riwayat dari JSON, pemanggil memberikan tanggal jatuh tempo yang sudah tersimpan. Karena itu, perubahan kebijakan lama pinjam kelak tidak mengubah kesepakatan transaksi lama.

Property `aktif` dihitung dari `tanggal_kembali is None`. Tidak ada attribute boolean kedua yang harus diperbarui bersamaan. Method `kembalikan()` memeriksa keadaan dan tanggal terlebih dahulu; tanggal kembali baru diisi jika seluruh syarat terpenuhi. Method `ke_data()` menyiapkan nilai yang bisa ditulis sebagai JSON, tanpa melakukan pekerjaan file di dalam model.

SATU ATURAN, SATU TEMPAT PEMERIKSAAN

Model memeriksa kebenaran satu object. Layanan memeriksa aturan yang melibatkan koleksi object. Misalnya, `Peminjaman` memeriksa tanggal kembali, tetapi hanya `Perpustakaan` yang dapat memeriksa apakah buku sudah mempunyai peminjaman aktif lain.

12.2 Encapsulation

Membatasi Perubahan melalui Antarmuka Object

Encapsulation pada proyek ini menjaga agar perubahan data mengikuti aturan yang sudah ditetapkan. Property `kode` dan `id_anggota` hanya mempunyai getter sehingga tidak dapat diberi nilai melalui antarmuka publik. Ini diperlukan karena kode dan ID dipakai sebagai kunci dictionary serta penghubung riwayat. Jika identitas berubah sepihak, kunci koleksi dan data object dapat tidak sesuai.

Property `judul` dan `nama` mempunyai setter karena keduanya boleh dikoreksi. Setter memanggil validasi sebelum mengisi attribute internal; jika nilai baru ditolak, nilai lama tetap tersimpan. Tanggal dan rujukan pada peminjaman hanya dibaca melalui property. Penyelesaian transaksi dilakukan dengan method `kembalikan()`, bukan dengan mengisi tanggal kembali secara langsung.

Setelah seluruh file inti tersedia, buat file `demo_encapsulation.py` di folder utama proyek. Isi dengan contoh berikut, lalu jalankan melalui terminal.

Perintah menjalankan demo

```
python demo_encapsulation.py
```

demo_encapsulation.py

```
from datetime import date

from perpustakaan import Anggota, BukuCetak, Peminjaman
from perpustakaan.kesalahan import DataTidakValidError, PeminjamanSelesaiError


buku = BukuCetak("B001", "Dasar OOP")
buku.judul = "OOP dengan Python"
print(buku)
try:
    buku.judul = "   "
except DataTidakValidError as exc:
    print("Ditolak:", exc)
print("Judul tetap:", buku.judul)
try:
    buku.kode = "B999"
except AttributeError:
    print("Kode hanya dapat dibaca.")

ani = Anggota("M001", "Ani")
pinjam = Peminjaman("P001", ani, buku, date(2026, 10, 27))
print("Aktif:", pinjam.aktif)
pinjam.kembalikan(date(2026, 11, 2))
print("Aktif setelah kembali:", pinjam.aktif)
try:
    pinjam.kembalikan(date(2026, 11, 3))
except PeminjamanSelesaiError as exc:
    print("Ditolak:", exc)
```

Keluaran

```
[B001] OOP dengan Python
Ditolak: Judul buku harus berupa teks yang terisi.
Judul tetap: OOP dengan Python
Kode hanya dapat dibaca.
Aktif: True
Aktif setelah kembali: False
Ditolak: Peminjaman sudah selesai.
```

Judul berhasil berubah karena nilai barunya valid. Percobaan mengisi judul dengan spasi ditolak sebelum `_judul` diubah. Pemberian nilai ke `buku.kode` memicu `AttributeError` karena property tersebut tidak mempunyai setter. Pemanggilan pengembalian kedua ditolak karena transaksi sudah selesai.

Menjaga Koleksi dan Memahami Batasnya

Layanan menyimpan buku, anggota, dan peminjaman dalam dictionary internal. Method daftar mengembalikan list baru. Menghapus isi list hasil tersebut tidak menghapus catatan di layanan. Namun, object di dalam list masih merupakan object yang sama, sehingga perubahan melalui setter yang sah tetap terlihat pada layanan.

Salinan koleksi tersebut adalah *shallow copy*, bukan salinan mendalam seluruh object. Awalan satu garis bawah juga merupakan konvensi untuk pemakaian internal, bukan mekanisme keamanan. Pemanggil yang sengaja menulis ke `_judul` atau koleksi internal dapat melewati aturan. Karena itu, kode antarmuka pada proyek ini selalu memakai property dan method publik yang disediakan.

PEMBUATAN OBJECT DAN TRANSAKSI APLIKASI

Pembuatan `Peminjaman` secara langsung pada contoh konsep hanya memeriksa aturan satu catatan. Dalam aplikasi, lakukan peminjaman melalui `perpus.pinjamkan(...)` agar keberadaan anggota, keberadaan buku, dan larangan peminjaman ganda ikut diperiksa. Demikian juga, lakukan pengembalian melalui layanan agar pemanggil bekerja dengan catatan yang terdaftar.

12.3 Inheritance/Composition sesuai Rancangan

Inheritance untuk Jenis Buku

`BukuCetak` dan `EBook` merupakan jenis `Buku`. Keduanya memakai validasi kode, setter judul, method `ke_data()`, dan representasi teks yang diwarisi dari induknya. Perbedaannya berada pada nilai `JENIS` dan implementasi `lama_pinjam()`.

Class `Buku` menggunakan `ABC` dan `@abstractmethod`. Kita tidak membuat `Buku(...)` secara langsung karena lama pinjamnya belum ditentukan. Class turunan yang konkret wajib menyediakan implementasi method abstrak tersebut. Pemilihan inheritance didasarkan pada kesamaan kontrak dan hubungan jenis, sebagaimana pembahasan pe-

makaian class oleh [Lutz, 2025].

Association untuk Rujukan, Composition untuk Catatan Layanan

`Peminjaman` bukan turunan `Anggota` ataupun `Buku`. Ia menyimpan rujukan ke keduanya melalui attribute internal. Hubungan ini adalah association: satu peminjaman menunjuk satu peminjam dan satu buku, sedangkan object anggota serta buku dapat dipakai oleh banyak catatan sepanjang riwayatnya.

Simpan contoh berikut sebagai `demo_relasi.py`. Operator `is` memeriksa apakah dua rujukan menunjuk object yang sama, bukan sekadar memiliki isi yang sama.

demo_relasi.py

```
from datetime import date

from perpustakaan import Anggota, Buku, BukuCetak, Peminjaman


ani = Anggota("M001", "Ani")
buku = BukuCetak("B001", "Dasar OOP")
pinjam = Peminjaman("P001", ani, buku, date(2026, 10, 27))

print("BukuCetak adalah Buku:", isinstance(buku, Buku))
print("Anggota yang sama:", pinjam.anggota is ani)
print("Buku yang sama:", pinjam.buku is buku)
ani.nama = "Ani Lestari"
print("Peminjam:", pinjam.anggota.nama)
```

Keluaran

```
BukuCetak adalah Buku: True
Anggota yang sama: True
Buku yang sama: True
Peminjam: Ani Lestari
```

Nama baru terlihat melalui `pinjam.anggota.nama` karena catatan menyimpan rujukan ke object `ani` yang sama. Nama anggota tidak digandakan di dalam peminjaman. Konsekuensinya, riwayat menampilkan nama anggota terbaru; menyimpan salinan nama pada waktu transaksi merupakan kebutuhan berbeda yang belum diterapkan pada proyek ini.

Dalam rancangan Bab 11, layanan `Perpustakaan` memiliki catatan peminjamannya sebagai composition. Implementasinya dilakukan dengan membuat `Peminjaman` di dalam `pinjamkan()` dan menyimpannya dalam dictionary layanan. Buku dan anggota didaftarkan sebagai object yang sudah dibuat, sesuai hubungan aggregation pada diagram. Composition di sini menyatakan kepemilikan dalam model aplikasi; menutup program tidak berarti menghapus riwayat yang sudah disimpan ke file.

Object penyimpanan diberikan ke constructor layanan. Class `Perpustakaan`

memakainya tanpa mewarisi class file JSON. Pemilihan hubungan ini mengikuti tanggung jawab object: layanan mengelola transaksi, sedangkan object penyimpanan mengurus file.

12.4 Polymorphism bila Diperlukan

Polymorphism diperlukan ketika pemanggil menjalankan operasi yang sama tetapi perilakunya dapat berbeda menurut jenis object. Pada aplikasi ini, perbedaannya nyata: buku cetak mempunyai masa pinjam 14 hari dan e-book 7 hari. Constructor `Peminjaman` cukup memanggil `buku.lama_pinjam()`, tanpa memilih rumus berdasarkan jenis buku.

Simpan dan jalankan `demo_polymorphism.py` berikut. Kedua object melewati baris pembuatan peminjaman yang sama, tetapi menghasilkan tanggal jatuh tempo berbeda.

demo_polymorphism.py

```
from datetime import date

from perpustakaan import Anggota, BukuCetak, EBook, Peminjaman


ani = Anggota("M001", "Ani")
koleksi = [BukuCetak("B001", "Dasar OOP"), EBook("E001", "Dasar Python")]
for nomor, buku in enumerate(koleksi, start=1):
    pinjam = Peminjaman(f"P{nomor:03d}", ani, buku, date(2026, 10, 27))
    print(buku.kode, buku.lama_pinjam(), "hari, jatuh tempo",
          pinjam.tanggal_jatuh_tempo)
```

Keluaran

```
B001 14 hari, jatuh tempo 2026-11-10
E001 7 hari, jatuh tempo 2026-11-03
```

Jika kelak ditambahkan jenis buku lain dengan masa pinjam berbeda, turunan baru menyediakan `lama_pinjam()`. Rumus di constructor peminjaman tetap dapat digunakan. Namun, integrasi jenis baru juga memerlukan pembaruan pilihan input dan pemetaan jenis pada pemuatan JSON. Polymorphism mengurangi percabangan pada perhitungan transaksi, bukan menghilangkan seluruh pekerjaan integrasi.

Kontrak `Penyimpanan` juga memberi peluang polymorphism: layanan memanggil `muat()` dan `simpan(data)` tanpa mengetahui cara penulisan file. Bab ini menyediakan satu implementasi, yaitu JSON. Implementasi lain dapat ditambahkan jika kebutuhan memang berubah; tidak perlu membuat banyak class penyimpanan hanya untuk memperbanyak turunan.

12.5 Module/Package

Menyusun Folder dan Arah Import

Module adalah satu file Python, sedangkan package mengelompokkan module yang berkaitan. Pada proyek ini kita memakai package biasa dengan file `__init__.py`. Struktur akhirnya sebagai berikut. Folder `data` dan file JSON dibuat ketika penyimpanan pertama berhasil.

Struktur folder proyek

```
proyek_perpustakaan/
|-- main.py
|-- demo_integrasi.py
|-- demo_encapsulation.py
|-- demo_relasi.py
|-- demo_polymorphism.py
|-- cek_kontrak.py
|-- perpustakaan/
|   |-- __init__.py
|   |-- kesalahan.py
|   |-- model.py
|   |-- penyimpanan.py
|   `-- layanan.py
`-- data/
    `-- perpustakaan.json
```

Tabel 12.2. Isi file dan ketergantungannya.

File	Isi	Memakai
`kesalahan.py`	Exception khusus	Exception bawaan Python
`model.py`	Class domain dan validasinya	Exception khusus, tanggal, ABC
`penyimpanan.py`	Kontrak dan adapter JSON	Exception khusus, JSON, path, file sementara
`layanan.py`	Koordinasi aplikasi	Model, kontrak penyimpanan, exception
`__init__.py`	Nama publik package	Model, layanan, penyimpanan, exception
`main.py`	Menu terminal	Nama publik package dan pustaka standar

Model tidak mengimpor layanan; penyimpanan tidak mengimpor menu; layanan tidak mengimpor `main.py`. Arah import tersebut mencegah ketergantungan berputar

dan memungkinkan model digunakan kembali ketika menu terminal diganti dengan GUI. Pemisahan komponen berdasarkan pekerjaan yang jelas juga memudahkan pemeriksaan perilaku setiap komponen [Wilson, 2024].

Menetapkan Antarmuka Package

Buat `perpustakaan/__init__.py` dengan isi berikut. File ini menyediakan nama yang paling sering dipakai oleh antarmuka. Exception khusus yang lebih rinci tetap dapat diimpor dari module `perpustakaan.kesalahan`.

perpustakaan/__init__.py

```python
from .model import Anggota, Buku, BukuCetak, EBook, Peminjaman
from .layanan import Perpustakaan
from .penyimpanan import PenyimpananJSON
from .kesalahan import PerpustakaanError
```

Tanda titik pada `from .model import ...` berarti import relatif dari package yang sama. Sebaliknya, `main.py` memakai `from perpustakaan import ...` karena berada di luar package. Jalankan aplikasi dari folder `proyek_perpustakaan`, bukan dengan menjalankan `layanan.py` secara langsung.

Perintah di terminal

```
python main.py
python demo_integrasi.py
```

Jika sistem menyediakan perintah `python3` atau `py` untuk interpreter yang digunakan, sesuaikan nama perintahnya. Tidak ada perintah `pip install` yang diperlukan untuk proyek ini. Import package hanya menyiapkan class; program menu dijalankan oleh blok `if __name__ == "__main__"` dalam `main.py`.

12.6 File Handling

Mengubah Object Menjadi Data JSON

JSON tidak menyimpan object Python beserta method dan rujukannya secara langsung. Karena itu, setiap model menyediakan `ke_data()` yang menghasilkan dictionary berisi teks, angka, atau `None`. Tanggal diubah menjadi teks ISO seperti `2026-10-27`. Nilai `None` menjadi `null` di JSON.

Satu data aplikasi memuat `versi`, nomor peminjaman berikutnya, daftar buku, daftar anggota, dan daftar peminjaman. Penanda versi membuat format yang belum didukung dapat ditolak secara jelas. Nomor berikutnya disimpan agar ID transaksi tetap berlanjut setelah aplikasi ditutup.

Contoh satu catatan peminjaman dalam JSON

```
{
  "id_pinjam": "P001",
  "id_anggota": "M001",
  "kode_buku": "B001",
  "tanggal_pinjam": "2026-10-27",
  "tanggal_jatuh_tempo": "2026-11-10",
  "tanggal_kembali": null
}
```

Catatan tersebut menyimpan identitas anggota dan buku, bukan salinan object atau hasil `id()`. Ketika dimuat, buku dan anggota harus dibuat terlebih dahulu. Layanan kemudian mencari object yang sesuai dengan ID pada catatan. Dengan urutan itu, peminjaman kembali terhubung ke object yang terdaftar, bukan ke salinan terpisah.

Membuat Adapter Penyimpanan JSON

Buat `perpustakaan/penyimpanan.py` berikut. `Protocol` menyatakan bentuk operasi yang diperlukan layanan. Anotasi tersebut membantu pembacaan kode dan pemeriksaan tipe; Python tidak otomatis memeriksa kepatuhan object pada saat runtime hanya karena parameter diberi anotasi.

perpustakaan/penyimpanan.py

```python
import json
import os
import tempfile
from pathlib import Path
from typing import Protocol

from .kesalahan import DataRusakError, PenyimpananError


class Penyimpanan(Protocol):
    def muat(self) -> dict: ...
    def simpan(self, data: dict) -> None: ...


class PenyimpananJSON:
    def __init__(self, path):
        self.path = Path(path)

    def muat(self):
        try:
            with self.path.open(encoding="utf-8") as sumber:
                data = json.load(sumber)
        except FileNotFoundError:
```

```
            raise  # layanan menentukan perilaku saat belum ada file
        except (json.JSONDecodeError, UnicodeError) as exc:
            raise DataRusakError("File JSON tidak dapat dibaca.") from exc
        except OSError as exc:
            raise PenyimpananError("Gagal membuka file data.") from exc
        if not isinstance(data, dict):
            raise DataRusakError("Isi utama JSON harus berupa object.")
        return data

    def simpan(self, data):
        sementara = None
        try:
            self.path.parent.mkdir(parents=True, exist_ok=True)
            with tempfile.NamedTemporaryFile(
                mode="w", encoding="utf-8", dir=self.path.parent,
                prefix=".perpustakaan-", suffix=".tmp", delete=False,
            ) as tujuan:
                sementara = Path(tujuan.name)
                json.dump(data, tujuan, ensure_ascii=False, indent=2)
            os.replace(sementara, self.path)
        except (OSError, TypeError, ValueError) as exc:
            raise PenyimpananError("Gagal menyimpan data.") from exc
        finally:
            if sementara is not None:
                try:
                    sementara.unlink(missing_ok=True)
                except OSError:
                    pass  # pembersihan tidak menutupi kegagalan utama
```

Method `muat()` bertugas membaca JSON dan memastikan nilai utamanya berupa dictionary. Pemeriksaan makna data, seperti ID anggota yang tidak ditemukan atau dua transaksi aktif untuk satu buku, menjadi tugas layanan. File yang belum ada dibiarkan menghasilkan `FileNotFoundError`; layanan akan menganggapnya sebagai pemakaian pertama. File yang ada tetapi rusak menghasilkan exception yang berbeda.

Saat menyimpan, program menulis file sementara pada folder yang sama, menutupnya, lalu memanggil `os.replace()` untuk mengganti file tujuan. Cara ini mengurangi risiko file lama terpotong ketika proses penulisan belum selesai. Pada Windows, file sementara perlu ditutup sebelum diganti. Bagian `finally` membersihkan file sementara yang tersisa tanpa menutupi penyebab kegagalan utama.

BEDAKAN FILE BELUM ADA DAN FILE RUSAK

File yang belum ada adalah keadaan wajar pada pemakaian pertama. File rusak harus dilaporkan agar pengguna dapat memeriksa atau memulihkan datanya. Aplikasi terminal pada bab ini berhenti jika pemuatan gagal; aplikasi tidak melanjutkan dengan data kosong lalu menimpa

simpanan lama.
Penggantian melalui file sementara bukan sistem pencadangan, bukan pengaturan akses serentak, dan tidak menjamin pemulihan dari setiap gangguan listrik atau kerusakan disk. Proyek ini tetap mengikuti batas satu petugas dan satu proses aplikasi.

Memuat Data tanpa Mengubah Keadaan Sebagian

Layanan tidak langsung mengosongkan koleksi ketika mulai memuat file. Ia membuat keadaan sementara, memeriksa seluruh buku, anggota, dan transaksi, lalu mengganti keadaan lama hanya setelah semua data sah. Jika satu rujukan tidak ditemukan atau satu tanggal salah, proses dibatalkan dan keadaan layanan yang lama tetap ada.

Tanggal jatuh tempo dibaca dari file dan diberikan ke constructor peminjaman. Tanggal kembali yang bukan `null` diterapkan melalui `kembalikan()` agar pemeriksaan tanggal dan status tetap digunakan. Setelah itu, layanan memeriksa ID peminjaman, larangan dua catatan aktif pada satu buku, serta urutan nomor berikutnya. Implementasi lengkap proses ini berada dalam `layanan.py` pada subbab berikut.

12.7 Integrasi Antar-Class

Menghubungkan Aturan melalui Layanan

Layanan menjadi pintu masuk operasi aplikasi. Menu tidak mencari sendiri transaksi aktif lalu mengubah attribute peminjaman. Menu memanggil `pinjamkan()` atau `kembalikan()` dan menangani hasilnya. Dengan begitu, GUI pada Bab 13 dapat menggunakan aturan yang sama.

Buat `perpustakaan/layanan.py` dengan isi lengkap berikut. Koleksi buku, anggota, dan peminjaman memakai dictionary dengan identitas sebagai kunci. List untuk tampilan dibuat melalui method daftar. JSON tetap memakai daftar record karena format penyimpanan tidak harus meniru bentuk koleksi internal secara persis.

perpustakaan/layanan.py

```python
from datetime import date

from .kesalahan import (
    PerpustakaanError, DataTidakValidError, DataTidakDitemukanError,
    BukuSedangDipinjamError, DataRusakError,
)
from .model import Anggota, Buku, BukuCetak, EBook, Peminjaman, teks_wajib
from .penyimpanan import Penyimpanan


class Perpustakaan:
```

```
    def __init__(self, penyimpanan: Penyimpanan):
        self._penyimpanan = penyimpanan
        self._buku = {}
        self._anggota = {}
        self._peminjaman = {}
        self._nomor_pinjam_berikutnya = 1

    def tambah_buku(self, buku):
        if not isinstance(buku, Buku):
            raise DataTidakValidError("Data harus berupa object Buku.")
        if buku.kode in self._buku:
            raise DataTidakValidError("Kode buku sudah terdaftar.")
        self._buku[buku.kode] = buku

    def tambah_anggota(self, anggota):
        if not isinstance(anggota, Anggota):
            raise DataTidakValidError("Data harus berupa object Anggota.")
        if anggota.id_anggota in self._anggota:
            raise DataTidakValidError("ID anggota sudah terdaftar.")
        self._anggota[anggota.id_anggota] = anggota

    def _cari(self, koleksi, identitas, label):
        identitas = teks_wajib(identitas, label)
        if identitas not in koleksi:
            raise DataTidakDitemukanError(f"{label} tidak ditemukan.")
        return koleksi[identitas]

    def tersedia(self, kode_buku):
        buku = self._cari(self._buku, kode_buku, "Buku")
        return not any(
            p.aktif and p.buku.kode == buku.kode
            for p in self._peminjaman.values()
        )

    def pinjamkan(self, id_anggota, kode_buku, tanggal):
        anggota = self._cari(self._anggota, id_anggota, "Anggota")
        buku = self._cari(self._buku, kode_buku, "Buku")
        if not self.tersedia(buku.kode):
            raise BukuSedangDipinjamError("Buku masih dipinjam.")
        identitas = f"P{self._nomor_pinjam_berikutnya:03d}"
        pinjam = Peminjaman(identitas, anggota, buku, tanggal)
        self._peminjaman[identitas] = pinjam
        self._nomor_pinjam_berikutnya += 1
        return pinjam

    def kembalikan(self, id_pinjam, tanggal):
```

```
        pinjam = self._cari(self._peminjaman, id_pinjam, "Peminjaman")
        pinjam.kembalikan(tanggal)

    def daftar_buku(self):
        return list(self._buku.values())

    def daftar_anggota(self):
        return list(self._anggota.values())

    def daftar_peminjaman(self, aktif_saja=False):
        return [p for p in self._peminjaman.values()
                if not aktif_saja or p.aktif]

    def simpan(self):
        data = {
            "versi": 1,
            "nomor_pinjam_berikutnya": self._nomor_pinjam_berikutnya,
            "buku": [b.ke_data() for b in self._buku.values()],
            "anggota": [a.ke_data() for a in self._anggota.values()],
            "peminjaman": [p.ke_data() for p in self._peminjaman.values()],
        }
        self._penyimpanan.simpan(data)

    def muat(self):
        try:
            data = self._penyimpanan.muat()
        except FileNotFoundError:
            return  # pertama kali dipakai: belum ada data di disk

        # Susun keadaan sementara; keadaan lama tetap utuh jika data salah.
        try:
            wajib = {"versi", "nomor_pinjam_berikutnya", "buku",
                     "anggota", "peminjaman"}
            if not isinstance(data, dict) or set(data) != wajib:
                raise DataTidakValidError("Field data tidak lengkap.")
            if type(data["versi"]) is not int or data["versi"] != 1:
                raise DataTidakValidError("Versi data tidak didukung.")
            for nama in ("buku", "anggota", "peminjaman"):
                if not isinstance(data[nama], list):
                    raise DataTidakValidError("Koleksi harus berupa list.")
            baru = Perpustakaan(self._penyimpanan)
            jenis_buku = {"cetak": BukuCetak, "ebook": EBook}
            for d in data["buku"]:
                kelas = jenis_buku[d["jenis"]]
                baru.tambah_buku(kelas(d["kode"], d["judul"]))
            for d in data["anggota"]:
```

```
                baru.tambah_anggota(Anggota(d["id_anggota"], d["nama"]))
            nomor_terbesar = 0
            for d in data["peminjaman"]:
                p = Peminjaman(
                    d["id_pinjam"], baru._anggota[d["id_anggota"]],
                    baru._buku[d["kode_buku"]],
                    date.fromisoformat(d["tanggal_pinjam"]),
                    date.fromisoformat(d["tanggal_jatuh_tempo"]),
                )
                if d["tanggal_kembali"] is not None:
                    p.kembalikan(date.fromisoformat(d["tanggal_kembali"]))
                identitas = p.id_pinjam
                if (not identitas.startswith("P")
                        or not identitas[1:].isdigit()
                        or identitas in baru._peminjaman):
                    raise DataTidakValidError("ID peminjaman tidak sah.")
                if p.aktif and not baru.tersedia(p.buku.kode):
                    raise DataTidakValidError("Buku dipinjam dua kali.")
                baru._peminjaman[identitas] = p
                nomor_terbesar = max(nomor_terbesar, int(identitas[1:]))
            nomor = data["nomor_pinjam_berikutnya"]
            if type(nomor) is not int or nomor <= nomor_terbesar:
                raise DataTidakValidError("Penomoran peminjaman tidak sah.")
            baru._nomor_pinjam_berikutnya = nomor
        except (KeyError, TypeError, ValueError, PerpustakaanError) as exc:
            raise DataRusakError("Struktur data perpustakaan tidak sah.") from
    exc

        self._buku = baru._buku
        self._anggota = baru._anggota
        self._peminjaman = baru._peminjaman
        self._nomor_pinjam_berikutnya = baru._nomor_pinjam_berikutnya
```

Method `pinjamkan()` mencari anggota dan buku yang terdaftar, memeriksa ketersediaan, lalu membentuk catatan. Penambahan ke dictionary dan kenaikan nomor dilakukan setelah constructor berhasil. Karena itu, permintaan yang ditolak tidak menambah riwayat dan tidak menghabiskan nomor transaksi.

Method `tersedia()` menghitung status dari peminjaman aktif. Setelah tanggal kembali terisi, catatan tetap berada di dictionary sebagai riwayat, tetapi tidak lagi menghalangi peminjaman berikutnya. Kita tidak perlu menyelaraskan flag `dipinjam` pada buku dengan flag `aktif` pada transaksi.

Pada `muat()`, pemetaan `jenis_buku` mengubah penanda `"cetak"` atau `"ebook"` menjadi class pembentuk object. Percabangan saat membaca format file berbeda tujuannya dari perhitungan lama pinjam. Setelah object terbentuk, transaksi kembali memakai polymorphism melalui `lama_pinjam()`.

Membuat Menu Terminal

Buat `main.py` di folder utama proyek. Path data dibangun dari lokasi file program, sehingga tidak bergantung pada folder kerja terminal. Menu menerima input, mengubah teks tanggal menjadi `date`, dan meneruskan pekerjaan ke layanan.

main.py

```python
from datetime import date
from pathlib import Path

from perpustakaan import (
    Anggota, BukuCetak, EBook, Perpustakaan,
    PenyimpananJSON, PerpustakaanError,
)


def main():
    path = Path(__file__).resolve().parent / "data" / "perpustakaan.json"
    perpus = Perpustakaan(PenyimpananJSON(path))
    try:
        perpus.muat()
    except PerpustakaanError as exc:
        print("Data tidak dimuat:", exc)
        return 1  # tidak melanjutkan dengan data kosong lalu menimpa file

    while True:
        print("\n1 Buku | 2 Tambah buku | 3 Tambah anggota")
        print("4 Pinjam | 5 Kembali | 6 Riwayat | 0 Simpan dan keluar")
        try:
            pilihan = input("Pilihan: ").strip()
            if pilihan == "1":
                for buku in perpus.daftar_buku():
                    status = "tersedia" if perpus.tersedia(buku.kode) else "
    dipinjam"
                    print(buku, "-", status)
            elif pilihan == "2":
                kode = input("Kode buku: ")
                judul = input("Judul: ")
                jenis = input("Jenis (cetak/ebook): ").strip().lower()
                kelas = {"cetak": BukuCetak, "ebook": EBook}.get(jenis)
                if kelas is None:
                    raise ValueError("Jenis harus cetak atau ebook.")
                perpus.tambah_buku(kelas(kode, judul))
                print("Buku terdaftar.")
            elif pilihan == "3":
                identitas = input("ID anggota: ")
                nama = input("Nama: ")
```

```
                perpus.tambah_anggota(Anggota(identitas, nama))
                print("Anggota terdaftar.")
            elif pilihan == "4":
                anggota = input("ID anggota: ")
                buku = input("Kode buku: ")
                tanggal = date.fromisoformat(input("Tanggal YYYY-MM-DD: ").
    strip())
                pinjam = perpus.pinjamkan(anggota, buku, tanggal)
                print("Berhasil:", pinjam.id_pinjam,
                      "jatuh tempo", pinjam.tanggal_jatuh_tempo)
            elif pilihan == "5":
                identitas = input("ID peminjaman: ")
                tanggal = date.fromisoformat(input("Tanggal YYYY-MM-DD: ").
    strip())
                perpus.kembalikan(identitas, tanggal)
                print("Pengembalian tercatat.")
            elif pilihan == "6":
                for p in perpus.daftar_peminjaman():
                    status = "aktif" if p.aktif else "selesai"
                    print(p.id_pinjam, p.anggota.nama, p.buku.kode, status)
            elif pilihan == "0":
                perpus.simpan()
                print("Data tersimpan.")
                return 0
            else:
                print("Pilihan tidak dikenal.")
        except (PerpustakaanError, ValueError) as exc:
            print("Ditolak:", exc)
        except (EOFError, KeyboardInterrupt):
            print("\nInput dihentikan; perubahan sesi ini belum disimpan.")
            return 1


if __name__ == "__main__":
    raise SystemExit(main())
```

Jalankan `python main.py`, lalu daftarkan B001 sebagai buku cetak berjudul Dasar OOP dan M001 sebagai anggota bernama Ani. Gunakan menu Pinjam dengan tanggal `2026-10-27`. Contoh berikut merupakan cuplikan interaksi; tampilan menu yang berulang tidak dituliskan kembali.

Cuplikan interaksi terminal

```
Pilihan: 4
ID anggota: M001
Kode buku: B001
Tanggal YYYY-MM-DD: 2026-10-27
```

```
Berhasil: P001 jatuh tempo 2026-11-10

Pilihan: 1
[B001] Dasar OOP – dipinjam

Pilihan: 0
Data tersimpan.
```

Perubahan selama menu berjalan berada di memori. Pilihan `0` menyimpan data dan baru keluar jika penyimpanan berhasil. Jika penyimpanan ditolak, program menampilkan pesan dan kembali ke menu sehingga pengguna dapat mencoba lagi. Penghentian dengan Ctrl+C atau berakhirnya input tidak menyimpan perubahan sesi secara otomatis. Buka kembali program untuk memeriksa bahwa data yang berhasil disimpan tetap terbaca.

Memeriksa Alur Terpadu secara Berulang

Menu berguna untuk mencoba aplikasi sebagai pengguna, tetapi pengulangan input manual mudah keliru. Buat `demo_integrasi.py` berikut untuk memeriksa satu alur tetap: daftar buku dan anggota, pinjam, tolak peminjaman kedua, kembalikan, pinjam e-book, simpan, lalu muat ulang.

Contoh memakai `TemporaryDirectory` agar setiap eksekusi dimulai dari keadaan baru. File demo berada di folder sementara dan dibersihkan saat blok selesai; file data milik `main.py` tidak digunakan.

demo_integrasi.py

```python
from datetime import date
from pathlib import Path
from tempfile import TemporaryDirectory

from perpustakaan import Anggota, BukuCetak, EBook, Perpustakaan,
    PenyimpananJSON
from perpustakaan.kesalahan import BukuSedangDipinjamError


with TemporaryDirectory() as folder:
    simpanan = PenyimpananJSON(Path(folder) / "perpustakaan.json")
    perpus = Perpustakaan(simpanan)
    perpus.muat()
    perpus.tambah_buku(BukuCetak("B001", "Dasar OOP"))
    perpus.tambah_buku(EBook("E001", "Dasar Python"))
    perpus.tambah_anggota(Anggota("M001", "Ani"))

    pinjam = perpus.pinjamkan("M001", "B001", date(2026, 10, 27))
    print("Pinjam:", pinjam.id_pinjam, "jatuh tempo", pinjam.
```

```
        tanggal_jatuh_tempo)
19      print("B001 tersedia:", perpus.tersedia("B001"))
20      try:
21          perpus.pinjamkan("M001", "B001", date(2026, 10, 28))
22      except BukuSedangDipinjamError as exc:
23          print("Ditolak:", exc)
24      print("Jumlah riwayat:", len(perpus.daftar_peminjaman()))
25
26      perpus.kembalikan("P001", date(2026, 11, 2))
27      print("Sesudah kembali:", perpus.tersedia("B001"))
28      digital = perpus.pinjamkan("M001", "E001", date(2026, 10, 27))
29      print("E-book jatuh tempo:", digital.tanggal_jatuh_tempo)
30      perpus.simpan()
31
32      dimuat = Perpustakaan(simpanan)
33      dimuat.muat()
34      print("Dimuat:", len(dimuat.daftar_buku()), "buku,",
35            len(dimuat.daftar_peminjaman()), "riwayat")
36      print("Status:", [(p.id_pinjam, p.aktif) for p in dimuat.daftar_peminjaman()
        ])
```

Keluaran

```
Pinjam: P001 jatuh tempo 2026-11-10
B001 tersedia: False
Ditolak: Buku masih dipinjam.
Jumlah riwayat: 1
Sesudah kembali: True
E-book jatuh tempo: 2026-11-03
Dimuat: 2 buku, 2 riwayat
Status: [('P001', False), ('P002', True)]
```

Jumlah riwayat tetap satu setelah percobaan peminjaman ganda, sehingga penolakan tidak membuat catatan tambahan. Setelah P001 dikembalikan, B001 tersedia kembali. Transaksi e-book berikutnya mendapat ID P002 dan jatuh tempo tujuh hari setelah tanggal pinjam. Pemuatan ulang mempertahankan kedua catatan beserta statusnya: P001 selesai, P002 aktif.

Memeriksa Kontrak dengan Assertion

Keluaran membantu pembaca menelusuri alur, tetapi assertion dapat memeriksa syarat secara langsung. Simpan pemeriksaan berikut sebagai `cek_kontrak.py` di folder utama. Contoh ini menilai relasi setelah simpan–muat dan memastikan pengembalian yang ditolak tidak mengubah transaksi. Assertion di sini dipakai untuk pengujian; validasi masukan aplikasi tetap menggunakan exception, sebab assertion dapat dinonaktifkan oleh opsi interpreter.

cek_kontrak.py

```python
from datetime import date
from pathlib import Path
from tempfile import TemporaryDirectory

from perpustakaan import Anggota, BukuCetak, Perpustakaan, PenyimpananJSON
from perpustakaan.kesalahan import DataTidakValidError


with TemporaryDirectory() as folder:
    simpanan = PenyimpananJSON(Path(folder) / "data.json")
    perpus = Perpustakaan(simpanan)
    perpus.tambah_buku(BukuCetak("B001", "Dasar OOP"))
    perpus.tambah_anggota(Anggota("M001", "Ani"))
    pinjam = perpus.pinjamkan("M001", "B001", date(2026, 10, 27))

    try:
        perpus.kembalikan("P001", date(2026, 10, 26))
    except DataTidakValidError:
        pass
    else:
        raise AssertionError("Tanggal kembali seharusnya ditolak.")
    assert pinjam.aktif and pinjam.tanggal_kembali is None
    assert len(perpus.daftar_peminjaman()) == 1

    perpus.simpan()
    dimuat = Perpustakaan(simpanan)
    dimuat.muat()
    riwayat = dimuat.daftar_peminjaman()
    assert riwayat[0].buku is dimuat.daftar_buku()[0]
    assert riwayat[0].anggota is dimuat.daftar_anggota()[0]
    assert not dimuat.tersedia("B001")

    dimuat.kembalikan("P001", date(2026, 11, 2))
    berikut = dimuat.pinjamkan("M001", "B001", date(2026, 11, 3))
    assert berikut.id_pinjam == "P002"

print("Kontrak pengembalian, relasi, dan nomor transaksi terpenuhi.")
```

Keluaran

```
Kontrak pengembalian, relasi, dan nomor transaksi terpenuhi.
```

Pemeriksaan identitas rujukan memakai `is`: object buku pada transaksi hasil muat harus sama dengan object yang ada di daftar buku layanan hasil muat. Kesamaan judul saja belum membuktikan hubungan tersebut. Pemeriksaan ID P002 memastikan penghitung ikut disimpan, bukan kembali ke satu saat aplikasi dibuka ulang.

Tabel 12.3. Skenario tambahan dan hasil yang harus diperiksa.

Percobaan	Hasil yang diharapkan
Kode buku atau ID anggota ganda	Ditolak; object yang sudah terdaftar tetap sama.
Anggota atau buku tidak terdaftar	Ditolak; riwayat dan nomor berikutnya tidak berubah.
Mengembalikan sebelum tanggal pinjam	Ditolak; transaksi tetap aktif.
Mengembalikan transaksi selesai	Ditolak; tanggal kembali sebelumnya tetap tersimpan.
File JSON berisi teks yang tidak sah	Pemuatan gagal dengan pesan; simpanan tidak ditimpa data kosong.
JSON sah tetapi rujukan ID hilang	Pemuatan gagal; keadaan lama di memori tetap utuh.
Dua catatan aktif untuk satu buku	Data ditolak saat dimuat.
Gagal menulis file sementara	Kesalahan penyimpanan dilaporkan; file tujuan lama belum diganti.

Urutan Pemeriksaan sebelum Membuat GUI

1. Lengkapi file package, `main.py`, dan file demo pada folder yang benar.
2. Jalankan demo konsep dan cocokkan keluarannya dengan contoh pada bab ini.
3. Jalankan `python demo_integrasi.py` serta `python cek_kontrak.py`.
4. Coba menu terminal dari data baru, simpan dengan pilihan 0, lalu buka kembali.
5. Periksa skenario penolakan. Catat keadaan sebelum dan sesudah operasi yang gagal.
6. Gunakan salinan file data untuk mencoba kasus kerusakan JSON atau rujukan ID.

Setelah langkah tersebut terpenuhi, model, layanan, dan penyimpanan sudah dapat dipakai oleh GUI. Pada Bab 13, tombol dan formulir akan menggantikan input menu sambil tetap memanggil operasi layanan yang sama.

12.8 Rangkuman

- Implementasi menerjemahkan tanggung jawab dan kontrak rancangan menjadi perilaku yang dapat dijalankan dan diperiksa.
- Property menjaga perubahan data: identitas hanya dibaca, sedangkan judul dan nama diperbarui melalui setter yang melakukan validasi.
- Inheritance dipakai untuk jenis buku; association menghubungkan transaksi dengan anggota serta buku; layanan memiliki kumpulan catatan peminjamannya.
- Polymorphism memungkinkan perhitungan jatuh tempo memanggil operasi yang sama untuk jenis buku yang berbeda.
- Module memisahkan model, exception, penyimpanan, layanan, dan tampilan. Arah import mengikuti tanggung jawab tersebut.
- JSON menyimpan data sederhana serta ID rujukan. Hubungan object dibangun kembali setelah buku dan anggota dimuat.
- Pemuatan divalidasi pada keadaan sementara sebelum mengganti keadaan lama. Penyimpanan menulis file sementara sebelum mengganti file tujuan.
- Aturan lintas-object dijalankan melalui layanan. Kegagalan transaksi tidak boleh meninggalkan perubahan sebagian.
- Pemeriksaan integrasi perlu mencakup hasil normal, penolakan, kelanjutan ID, dan hubungan object setelah simpan–muat.

12.9 Kuis Teoritis

Pilihan Ganda

Pilihlah satu jawaban yang paling tepat berdasarkan implementasi dalam bab ini.

1. Kode buku disediakan sebagai property hanya-baca karena ...
   a. judul buku tidak boleh diubah
   b. kode menjadi identitas tetap dan kunci koleksi
   c. semua attribute Python harus tidak dapat diubah
   d. JSON tidak dapat menyimpan kode berupa teks
2. Jika setter judul menerima teks berisi spasi saja, hasilnya adalah ...
   a. judul lama dihapus sebelum kesalahan dilaporkan
   b. judul diganti dengan satu spasi
   c. exception dilaporkan dan judul lama tetap ada
   d. object buku otomatis dihapus
3. Pemanggilan `buku.lama_pinjam()` pada constructor peminjaman merupakan contoh ...
   a. polymorphism
   b. pembacaan file
   c. duplikasi seluruh object
   d. penghapusan inheritance
4. Pemeriksaan bahwa B001 belum mempunyai peminjaman aktif lain dilakukan oleh ...
   a. menu saja
   b. `Anggota.info()`
   c. encoder JSON secara otomatis
   d. layanan `Perpustakaan`
5. Setelah `daftar_buku()` menghasilkan list baru, pemanggilan `clear()` pada list itu akan ...
   a. menghapus semua buku pada layanan
   b. mengosongkan list hasil tanpa menghapus dictionary layanan
   c. menghapus file JSON secara otomatis
   d. mengubah semua kode buku
6. Cara menyimpan hubungan peminjaman dengan buku dalam JSON adalah ...
   a. menyalin method buku sebagai teks kode
   b. menyimpan alamat memori Python

c. menyimpan kode buku lalu menghubungkannya kembali saat dimuat
d. menyimpan hanya judul dan mengabaikan identitas

7. Alasan memakai keadaan sementara saat pemuatan adalah ...

a. mencegah kegagalan di tengah pemuatan meninggalkan data lama yang sudah berubah sebagian
b. membuat setiap kesalahan selalu diabaikan
c. menghindari kebutuhan validasi
d. mengganti semua ID dengan `id()`

8. Saat buku yang masih dipinjam diminta lagi, nomor berikutnya ...

a. selalu bertambah dua
b. selalu kembali ke satu
c. berubah menjadi kode anggota
d. tetap karena catatan baru belum berhasil dibuat

9. Jika JSON ada tetapi tidak dapat dibaca, menu terminal akan ...

a. menghapusnya tanpa memberi pesan
b. melaporkan kegagalan dan berhenti sebelum mengubah simpanan
c. otomatis menganggap semua buku tersedia lalu menyimpan
d. memuat hanya baris pertama tanpa pemeriksaan

10. Penyimpanan tanggal jatuh tempo di dalam transaksi bertujuan untuk ...

a. selalu mengikuti perubahan kebijakan terbaru pada riwayat lama
b. mengganti tanggal pinjam setiap kali file dibuka
c. mempertahankan kesepakatan jatuh tempo ketika transaksi dibuat
d. menghilangkan kebutuhan tanggal kembali

Uraian

1. Jelaskan mengapa validasi tanggal diletakkan pada `Peminjaman`, sedangkan pemeriksaan buku tersedia diletakkan pada `Perpustakaan`.
2. Telusuri apa yang berubah dan tidak berubah ketika peminjaman kedua untuk buku yang sama ditolak.
3. Jelaskan perbedaan inheritance buku dengan association antara peminjaman dan anggota. Gunakan hasil `demo_relasi.py` untuk mendukung jawaban.
4. Mengapa pemuatan JSON perlu membentuk buku dan anggota sebelum peminjaman? Apa yang harus dilakukan jika ID rujukannya tidak ditemukan?
5. Jika ditambahkan `BukuReferensi` dengan masa pinjam tiga hari, bagian mana yang harus ditambah dan bagian mana yang tetap dapat digunakan? Sertakan pemetaan JSON serta pilihan input dalam jawaban.

KUNCI PILIHAN GANDA

1 b; 2 c; 3 a; 4 d; 5 b; 6 c; 7 a; 8 d; 9 b; 10 c.

12.10 Praktik Mandiri

Latihan berikut melanjutkan proyek, bukan mengganti seluruh rancangan. Gunakan salinan proyek dan data khusus latihan. Bentuk latihan dengan syarat serta hasil yang dapat diperiksa membantu pembaca menilai perilaku programnya sendiri, sebagaimana pendekatan latihan pada [Edcorner Learning, tt].

Praktik 1: Menambah Jenis Buku

Tambahkan class `BukuReferensi` dengan penanda jenis `"referensi"` dan lama pinjam tiga hari. Perbarui ekspor package, pemetaan jenis ketika memuat, serta pilihan input menu. Gunakan seluruh validasi kode dan judul yang sudah tersedia pada induk.

- Hasil wajib: peminjaman pada 27 Oktober 2026 jatuh tempo pada 30 Oktober 2026.
- Simpan, muat ulang, lalu pastikan object kembali menjadi `BukuReferensi`, bukan `BukuCetak`.
- Buktikan bahwa rumus pembuatan jatuh tempo pada `Peminjaman` tidak perlu ditambah percabangan jenis.

Praktik 2: Menampilkan Peminjaman Terlambat

Tambahkan operasi layanan yang menerima tanggal pemeriksaan dan menghasilkan peminjaman yang masih aktif dengan tanggal jatuh tempo *sebelum* tanggal pemeriksaan. Jangan menyimpan flag `terlambat` tambahan karena status ini bergantung pada tanggal pemeriksaan.

- Hasil wajib: transaksi yang jatuh tempo 10 November belum terlambat pada 10 November, tetapi terlambat pada 11 November jika belum dikembalikan.
- Transaksi selesai tidak ditampilkan, walaupun tanggal pemeriksaan melewati jatuh temponya.
- Tambahkan menu laporan dan uji masukan tanggal yang tidak valid tanpa mengubah riwayat.

Praktik 3: Memeriksa Kegagalan Simpan–Muat

Buat pemeriksaan terpisah menggunakan folder sementara. Siapkan data sah, lalu uji salinan data dengan ID anggota hilang, dua transaksi aktif pada satu buku, nomor berikutnya yang sudah terpakai, dan JSON yang tidak sah. Periksa exception serta keadaan layanan sebelum dan sesudah setiap percobaan.

- Hasil wajib: seluruh data salah ditolak, keadaan lama tetap ada, dan file asli tidak ditimpa oleh data kosong.
- Tambahkan kasus simpan–muat yang berhasil untuk membuktikan bahwa pemeriksaan tidak hanya menolak semua file.
- Catat langkah, hasil yang diharapkan, hasil aktual, dan penyebab jika terjadi perbedaan.

Tabel 12.4. Rubrik praktik implementasi.

Aspek	Bobot	Bukti
Aturan dan hasil operasi	35%	Skenario normal serta penolakan menghasilkan keadaan yang tepat.
Pemakaian konsep OOP	25%	Tanggung jawab, property, relasi, dan polymorphism sesuai kebutuhan.
Integritas simpan–muat	25%	Identitas, relasi, jenis object, tanggal, dan nomor transaksi bertahan.
Keterbacaan dan pemeriksaan	15%	File tersusun jelas, hasil bisa diulang, dan laporan menjelaskan perilaku yang diperiksa.

Bab 13

Implementasi GUI

Memberi tampilan pada aplikasi yang sudah dibangun.

13

DI BAB INI

- Implementasi GUI
- Komponen GUI
- Event handling
- Integrasi GUI dengan class OOP

Pada Bab 12, petugas perpustakaan sudah dapat mendaftarkan buku dan anggota, mencatat peminjaman, mengembalikan buku, serta menyimpan data melalui menu terminal. Sekarang kita menyediakan pekerjaan yang sama melalui jendela, formulir, tombol, dan tabel. Class domain serta layanan tetap menjadi dasar aplikasi; yang kita tambahkan adalah cara pengguna berinteraksi dengannya.

Bab ini melanjutkan dasar Tkinter pada Bab 10 dan rancangan aplikasi pada Bab 11. Hasilnya adalah GUI Sistem Peminjaman Perpustakaan dengan tiga tab: Buku, Anggota, dan Peminjaman. Seluruh kode tambahan ditempatkan dalam satu file `gui.py` agar hubungan antarkomponen mudah ditelusuri sebelum tampilan dipecah menjadi beberapa module.

CAPAIAN PEMBELAJARAN

Setelah mempelajari bab ini, Anda diharapkan mampu:

- mengubah kebutuhan aplikasi menjadi susunan formulir dan tabel;
- memilih widget dan mengatur hubungan widget dengan data masukan;
- menulis event handler yang membaca input, memanggil layanan, dan memperbarui tampilan;
- menghubungkan GUI dengan package OOP tanpa menggandakan aturan transaksi;
- menangani kegagalan operasi, penyimpanan, dan penutupan jendela;
- memeriksa kesesuaian data di layar dengan keadaan layanan serta file simpanan.

13.1 Implementasi GUI

Menentukan Pekerjaan Pengguna

Implementasi GUI dimulai dengan pekerjaan yang ingin diselesaikan pengguna. Petugas membutuhkan tempat untuk mengisi data, melihat hasil pendaftaran, memilih buku yang tersedia, dan mengetahui apakah peminjaman sudah selesai. Tombol dan tabel dipilih untuk mendukung pekerjaan tersebut.

Tab Buku memperlihatkan jenis dan status ketersediaan buku. Tab Anggota memuat ID dan nama anggota. Tab Peminjaman menampilkan tanggal pinjam, jatuh tempo, tanggal kembali, dan status transaksi. Pilihan buku pada formulir hanya memuat kode yang tersedia. Layanan tetap memeriksa ketersediaan ketika transaksi diminta.

Tabel 13.1. Pemetaan kebutuhan ke bagian antarmuka.

Kebutuhan	Bagian GUI	Operasi layanan
Mendaftarkan buku	Formulir kode, judul, dan jenis pada tab Buku	`tambah_buku()`
Mendaftarkan anggota	Formulir ID dan nama pada tab Anggota	`tambah_anggota()`
Meminjam buku	Pilihan anggota, buku tersedia, tanggal, dan tombol Pinjam	`pinjamkan()`
Mengembalikan buku	Pilihan baris riwayat, tanggal, dan tombol pengembalian	`kembalikan()`
Melihat keadaan	Tabel buku, anggota, dan seluruh riwayat	Method daftar dan `tersedia()`
Menyimpan data	Tombol Simpan dan konfirmasi saat keluar	`simpan()`

Menyiapkan Proyek dan Tkinter

Gunakan proyek Bab 12 yang sudah lengkap dan berhasil dijalankan. Tambahkan `gui.py` sejajar dengan `main.py`; isi lengkap file tersebut terdapat pada Subbab Integrasi GUI dengan Class OOP. Package `perpustakaan` tetap dipakai melalui antarmuka publiknya.

Struktur file yang digunakan

```
proyek_perpustakaan/
|-- main.py
|-- gui.py
|-- perpustakaan/
|   |-- __init__.py
|   |-- kesalahan.py
|   |-- model.py
|   |-- penyimpanan.py
|   `-- layanan.py
`-- data/
    `-- perpustakaan.json
```

Gunakan Python 3.10 atau yang lebih baru dengan dukungan Tkinter. Periksa dukungannya dari terminal dengan perintah berikut; sebuah jendela demonstrasi akan terbuka jika Tkinter tersedia. Tutup jendela itu sebelum menjalankan aplikasi.

Memeriksa Tkinter

```
python -m tkinter
```

Tkinter merupakan antarmuka Python untuk Tcl/Tk, tetapi dapat tidak disertakan

pada distribusi Python tertentu. Jika module tidak ditemukan atau jendela tidak dapat dibuka, lengkapi dukungan Tkinter sesuai petunjuk distribusi Python dan jalankan pada lingkungan desktop. Rujukan resmi tersedia pada dokumentasi Tkinter Python.

SATU APLIKASI UNTUK SATU FILE DATA

Versi terminal dan GUI menggunakan lokasi data yang sama, yaitu folder `data` di samping file program. Jalankan salah satunya dalam satu waktu. Aplikasi belum mengatur perubahan serentak dari beberapa proses; dua jendela yang menyimpan data berbeda dapat saling menimpa hasil simpanan. Gunakan salinan proyek untuk latihan agar data percobaan mudah dimulai ulang.

Menentukan Alur Kerja Jendela

Saat aplikasi dimulai, data dimuat sebelum jendela utama ditampilkan. Jika belum ada file, layanan memulai keadaan kosong. Jika file ada tetapi rusak atau tidak dapat dibaca, GUI menampilkan pesan dan berhenti. Keputusan ini mengikuti perilaku aplikasi terminal pada Bab 12.

Selama aplikasi berjalan, perubahan berada di memori. Tombol Simpan menulis keadaan terbaru ke file. Setelah perubahan yang berhasil, teks status mengingatkan bahwa data belum disimpan. Jika pengguna menutup jendela dalam keadaan tersebut, aplikasi menawarkan pilihan menyimpan, keluar tanpa menyimpan, atau membatalkan penutupan.

13.2 Komponen GUI

Memilih Widget sesuai Tugasnya

Contoh ini memakai `tkinter.ttk` untuk widget bertema. Tampilan detail tombol, tab, dan tabel dapat mengikuti sistem operasi. Kita mengatur susunan serta perilakunya melalui kode, tanpa bergantung pada kesamaan warna atau bentuk di setiap komputer.

`StringVar` menghubungkan nilai teks dengan widget melalui opsi `textvariable`. Method `get()` membaca isinya, sedangkan `set()` memperbaruinya. Nilai tanggal di `StringVar` masih berupa teks; GUI mengubahnya menjadi `date` sebelum memanggil layanan.

Menyusun Formulir dan Tabel

Jendela dibagi menjadi judul, area tab yang dapat membesar, serta baris status dan tombol di bagian bawah. Di dalam setiap tab, formulir berada di atas tabel. `pack()` mengatur susunan antarkelompok, sedangkan `grid()` mengatur kolom formulir dan hubungan tabel dengan scrollbar. Keduanya digunakan pada parent yang berbeda.

Pengaturan `fill="both"` dan `expand=True` memberi ruang tambahan kepada area tabel. Pada parent yang memakai `grid()`, bobot baris dan kolom ditetapkan dengan

Tabel 13.2. Widget dan perannya pada aplikasi perpustakaan.

Komponen	Pemakaian
`Tk`	Satu jendela utama yang menjalankan event loop.
`ttk.Notebook`	Mengelompokkan pekerjaan ke tab Buku, Anggota, dan Peminjaman.
`ttk.Frame`	Menampung formulir, tabel, atau baris tombol.
`ttk.Label`	Memberi nama kolom isian, petunjuk, judul, dan pesan status.
`ttk.Entry`	Menerima teks kode, judul, ID, nama, dan tanggal transaksi.
`ttk.Combobox`	Memilih jenis buku, ID anggota terdaftar, dan kode buku tersedia.
`ttk.Button`	Menjalankan pendaftaran, peminjaman, pengembalian, penyimpanan, atau penutupan.
`ttk.Treeview`	Menampilkan data dalam baris dan kolom serta memilih satu catatan.
`ttk.Scrollbar`	Menggeser tabel secara vertikal dan horizontal.
`messagebox`	Melaporkan penolakan atau meminta keputusan sebelum menutup.

`rowconfigure()` serta `columnconfigure()`. Scrollbar tetap tersedia ketika isi tabel lebih panjang atau lebih lebar daripada area yang terlihat.



Gambar 13.1. Skema tab Peminjaman setelah B001 dipinjam. B001 tidak lagi muncul sebagai pilihan buku tersedia. Gambar menunjukkan susunan informasi; bentuk widget sebenarnya mengikuti tema Tkinter pada komputer pembaca.

Memakai Identitas yang Tetap pada Pilihan

Combobox anggota menyimpan ID anggota, dan combobox buku menyimpan kode buku. Nama anggota dan judul dapat dilihat pada tab masing-masing. Pemakaian ID menghindari masalah ketika dua anggota bernama sama atau dua eksemplar buku berjudul sama. Program tidak perlu memotong teks label untuk menebak identitas object.

Pada `Treeview`, setiap baris diberi `iid` yang sama dengan identitas domainnya. Misalnya, baris peminjaman P001 memakai `iid="P001"`. Hasil `selection()` dapat langsung digunakan sebagai ID ketika memanggil layanan. Posisi baris tidak dijadikan ID karena urutannya dapat berubah.

Combobox dibuat dengan `state="readonly"` agar pengguna memilih dari daftar yang disediakan. Tabel memakai `selectmode="browse"` agar hanya satu catatan dipilih untuk pengembalian. Widget tersebut membantu masukan pengguna; aturan tentang keabsahan ID dan keadaan transaksi tetap diperiksa oleh layanan.

13.3 Event Handling

Menghubungkan Tindakan dengan Method

Event handler pada aplikasi berupa method dari object `AplikasiPerpustakaan`. Tombol menerima rujukan method melalui `command`, sedangkan event keyboard atau perubahan pilihan didaftarkan melalui `bind()`. Pendaftaran dilakukan sekali ketika widget dibuat; Tkinter memanggil handler ketika event yang sesuai terjadi.

Tabel 13.3. Event dan handler pada GUI.

Tindakan pengguna	Handler	Hasil
Klik Daftarkan buku	`tambah_buku()`	Membentuk buku sesuai jenis dan mendaftarkannya.
Klik Daftarkan anggota	`tambah_anggota()`	Membentuk anggota dan mendaftarkannya.
Klik Pinjam	`pinjam()`	Membaca pilihan serta tanggal, lalu meminta transaksi baru.
Klik Kembalikan pilihan	`kembalikan()`	Menggunakan ID baris yang dipilih untuk pengembalian.
Pilih baris pada tabel buku	`pilih_buku(event)`	Mengisi pilihan buku jika kode tersebut tersedia.
Klik Simpan atau tekan Ctrl+S	`simpan( event=None)`	Menyimpan data dan memperbarui teks status.
Klik Tutup atau tombol X	`tutup()`	Memeriksa perubahan yang belum disimpan.

RUJUKAN METHOD, BUKAN HASIL PEMANGGILANNYA

Gunakan `command=self.pinjam`, tanpa tanda kurung. Bentuk `command=self.pinjam()` menjalankan method ketika tombol dibuat dan menyerahkan hasilnya sebagai callback. Handler dari `bind()` menerima object event; parameter `event=None` memungkinkan handler tertentu dipakai oleh tombol maupun event keyboard.

Membaca Input dan Menangani Penolakan

Satu operasi GUI mengikuti urutan: baca input, ubah format bila diperlukan, panggil layanan, lalu perbarui tampilan jika berhasil. Fungsi `baca_tanggal()` menerima teks seperti `2026-10-27` dan menghasilkan object `date`. Format yang salah atau tanggal yang tidak ada, seperti 30 Februari, ditolak dengan pesan yang menjelaskan bentuk input yang benar.

Validasi format termasuk pekerjaan antarmuka. Validasi aturan seperti tanggal kembali yang mendahului tanggal pinjam tetap berada pada model. Dengan pembagian ini, kesalahan input dapat dijelaskan kepada pengguna tanpa memindahkan aturan peminjaman ke handler.

Method bantuan `_jalankan()` menerima sebuah fungsi operasi dan pesan keberhasilan. Ia menangkap `PerpustakaanError` serta `ValueError`, menampilkan kotak pesan jika operasi ditolak, lalu mengembalikan `False`. Jika operasi berhasil, ia menandai adanya perubahan, menyegarkan tampilan, dan mengembalikan `True`. Formulir pendaftaran hanya dikosongkan setelah hasilnya `True`, sehingga input yang ditolak masih dapat dikoreksi.

`_jalankan()` tidak menangkap seluruh `Exception`. Kesalahan pemrograman yang belum dikenali perlu diperbaiki, bukan disembunyikan sebagai masukan pengguna yang salah. Handler juga tidak memanggil `input()` atau membuat perulangan `while True`; setelah selesai, kendali kembali ke event loop.

Menyegarkan Tampilan setelah Keadaan Berubah

Method `segarkan()` membaca daftar terbaru dari layanan, mengisi ulang tabel, lalu memperbarui pilihan anggota dan buku tersedia. Jika sebuah kode buku sudah tidak tersedia, pilihan lama untuk kode tersebut dikosongkan. Pemilihan baris yang masih ada dipertahankan melalui `iid`.

Contohnya, setelah P001 dibuat, tabel riwayat menampilkan P001 sebagai aktif, tabel buku menampilkan B001 sebagai dipinjam, dan combobox tidak lagi menawarkan B001. Setelah P001 dikembalikan, riwayat tetap ada dengan status selesai dan B001 kembali tersedia. Semua perubahan layar berasal dari keadaan layanan yang sama.

Menyimpan dan Menutup dengan Keputusan yang Jelas

Attribute `berubah` menandai adanya operasi yang berhasil sejak penyimpanan terakhir. Ini adalah keadaan antarmuka, berbeda dari property `aktif` pada transaksi. Memilih baris atau mengganti isi formulir yang belum dikirim tidak mengubah data domain dan tidak menandai perubahan yang perlu disimpan.

Ketika `simpan()` berhasil, penanda menjadi `False`. Ketika penyimpanan gagal, penanda tetap seperti sebelumnya dan jendela tetap terbuka. Tombol X dihubungkan ke `tutup()` melalui protokol `WM_DELETE_WINDOW`, sehingga pemeriksaan tersebut berlaku juga ketika pengguna menutup lewat bingkai jendela.

`askyesnocancel()` menghasilkan `True`, `False`, atau `None`. Pemeriksaan `jawaban is None` membedakan Batal dari Tidak. Menyatukan keduanya dengan `if not jawaban` akan kehilangan perbedaan keputusan tersebut.

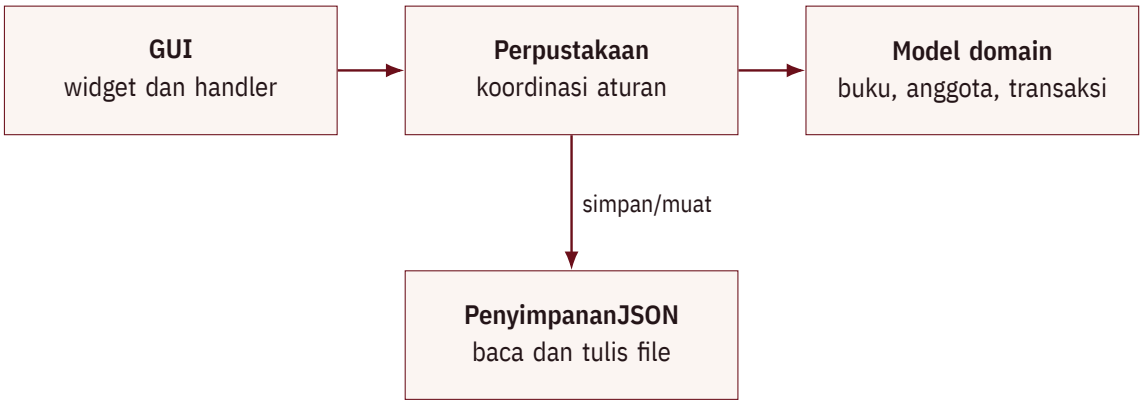
Tabel 13.4. Keputusan saat menutup jendela dengan perubahan belum tersimpan.

Pilihan	Perilaku aplikasi
Ya	Menyimpan terlebih dahulu; jendela ditutup hanya jika penyimpanan berhasil.
Tidak	Menutup tanpa menyimpan perubahan sesi tersebut. File simpanan sebelumnya tetap digunakan pada pembukaan berikutnya.
Batal	Membatalkan penutupan dan kembali ke jendela dengan keadaan yang sama.

13.4 Integrasi GUI dengan Class OOP

Memisahkan Tampilan, Layanan, dan Penyimpanan

Class `AplikasiPerpustakaan` menerima object jendela dan object layanan melalui constructor. Class ini memiliki rujukan ke keduanya serta menyimpan widget dan variabel formulir sebagai attribute. Class domain tidak mewarisi widget dan tidak mengimpor `tkinter`.



Gambar 13.2. GUI memakai layanan yang sama dengan menu terminal. Tanda panah menyatakan arah pemakaian komponen; bukan relasi inheritance.

GUI membuat object `BukuCetak`, `EBook`, atau `Anggota` dari formulir, kemudian mendaftarkannya lewat layanan. Peminjaman baru dibuat oleh layanan melalui `pinjamkan()`, sesuai rancangan composition pada Bab 11. GUI membaca property transaksi untuk menampilkan hasil, tanpa menulis ke dictionary atau attribute internal model.

Pemisahan tersebut memungkinkan pemeriksaan layanan Bab 12 tetap dipakai ketika antarmuka berubah. Pendekatan menyusun dan memeriksa komponen secara bertahap juga dibahas oleh [Wilson, 2024]. Pada bab ini, pemeriksaan GUI ditambahkan untuk menilai hubungan input, callback, dan tampilan yang tidak tercakup oleh pemeriksaan layanan saja.

Kode Lengkap Antarmuka

Buat `gui.py` di folder utama proyek, lalu salin **seluruh listing berikut sebagai isi file tersebut**. Seluruh helper widget, handler, dan fungsi awal aplikasi sudah disertakan. Tidak ada perubahan yang diperlukan pada package `perpustakaan` dari Bab 12.

gui.py

```
from datetime import date
from pathlib import Path
import tkinter as tk
from tkinter import messagebox, ttk

from perpustakaan import (
    Anggota, BukuCetak, EBook, Perpustakaan,
    PerpustakaanError, PenyimpananJSON,
)


def baca_tanggal(teks):
    teks = teks.strip()
    try:
        tanggal = date.fromisoformat(teks)
        if tanggal.isoformat() != teks:
            raise ValueError
    except ValueError as exc:
        raise ValueError("Gunakan tanggal sah berformat YYYY-MM-DD.") from exc
    return tanggal


class AplikasiPerpustakaan:
    def __init__(self, root, perpus):
        self.root = root
        self.perpus = perpus
        self.berubah = False
        root.title("Sistem Peminjaman Perpustakaan")
        root.geometry("1100x760")
        root.minsize(950, 650)

        self.kode = tk.StringVar(root)
        self.judul = tk.StringVar(root)
        self.jenis = tk.StringVar(root, value="cetak")
        self.id_anggota = tk.StringVar(root)
        self.nama = tk.StringVar(root)
        self.anggota_dipilih = tk.StringVar(root)
        self.buku_dipilih = tk.StringVar(root)
        self.tanggal = tk.StringVar(root, value=date.today().isoformat())
        self.status = tk.StringVar(root, value="Siap. Belum ada perubahan.")
```

```
41
42          ttk.Label(root, text="Sistem Peminjaman Perpustakaan",
43                    font=("TkDefaultFont", 16, "bold")).pack(pady=(12, 4))
44          self.tab = ttk.Notebook(root)
45          self.tab.pack(fill="both", expand=True, padx=12, pady=8)
46          self._buat_tab_buku()
47          self._buat_tab_anggota()
48          self._buat_tab_pinjam()
49
50          bawah = ttk.Frame(root, padding=(12, 0, 12, 12))
51          bawah.pack(fill="x")
52          ttk.Label(bawah, textvariable=self.status).pack(side="left")
53          ttk.Button(bawah, text="Tutup", command=self.tutup).pack(side="right")
54          ttk.Button(bawah, text="Simpan", command=self.simpan).pack(
55              side="right", padx=8)
56          root.bind("<Control-s>", self.simpan)
57          root.protocol("WM_DELETE_WINDOW", self.tutup)
58          self.segarkan()
59
60      def _halaman(self, judul):
61          halaman = ttk.Frame(self.tab, padding=12)
62          self.tab.add(halaman, text=judul)
63          return halaman
64
65      def _isian(self, parent, label, variabel, kolom, lebar=18):
66          ttk.Label(parent, text=label).grid(
67              row=0, column=kolom, sticky="w", pady=(0, 4))
68          isian = ttk.Entry(parent, textvariable=variabel, width=lebar)
69          isian.grid(row=1, column=kolom, sticky="ew", padx=(0, 8))
70          parent.columnconfigure(kolom, weight=1)
71          return isian
72
73      def _tabel(self, parent, kolom):
74          wadah = ttk.Frame(parent)
75          wadah.pack(fill="both", expand=True, pady=(12, 0))
76          wadah.rowconfigure(0, weight=1)
77          wadah.columnconfigure(0, weight=1)
78          nama = [k[0] for k in kolom]
79          tabel = ttk.Treeview(wadah, columns=nama, show="headings",
80                               selectmode="browse")
81          for nama, judul, lebar in kolom:
82              tabel.heading(nama, text=judul)
83              tabel.column(nama, width=lebar, minwidth=60, anchor="w")
84          vertikal = ttk.Scrollbar(wadah, orient="vertical", command=tabel.yview)
85          horizontal = ttk.Scrollbar(wadah, orient="horizontal", command=tabel.
    xview)
```

```
        tabel.configure(yscrollcommand=vertikal.set,
                        xscrollcommand=horizontal.set)
        tabel.grid(row=0, column=0, sticky="nsew")
        vertikal.grid(row=0, column=1, sticky="ns")
        horizontal.grid(row=1, column=0, sticky="ew")
        return tabel

    def _buat_tab_buku(self):
        halaman = self._halaman("Buku")
        form = ttk.Frame(halaman)
        form.pack(fill="x")
        self._isian(form, "Kode buku", self.kode, 0, 12)
        self._isian(form, "Judul", self.judul, 1, 36)
        ttk.Label(form, text="Jenis").grid(row=0, column=2, sticky="w")
        ttk.Combobox(form, textvariable=self.jenis, values=("cetak", "ebook"),
                     state="readonly", width=12).grid(row=1, column=2, padx=8)
        ttk.Button(form, text="Daftarkan buku", command=self.tambah_buku).grid(
            row=1, column=3)
        self.tabel_buku = self._tabel(halaman, [
            ("kode", "Kode", 100), ("judul", "Judul", 380),
            ("jenis", "Jenis", 100), ("status", "Status", 110),
        ])
        self.tabel_buku.bind("<<TreeviewSelect>>", self.pilih_buku)

    def _buat_tab_anggota(self):
        halaman = self._halaman("Anggota")
        form = ttk.Frame(halaman)
        form.pack(fill="x")
        self._isian(form, "ID anggota", self.id_anggota, 0, 14)
        self._isian(form, "Nama", self.nama, 1, 40)
        ttk.Button(form, text="Daftarkan anggota", command=self.tambah_anggota).
    grid(
            row=1, column=2)
        self.tabel_anggota = self._tabel(halaman, [
            ("id", "ID anggota", 130), ("nama", "Nama", 500),
        ])

    def _buat_tab_pinjam(self):
        halaman = self._halaman("Peminjaman")
        form = ttk.Frame(halaman)
        form.pack(fill="x")
        for kolom, label in enumerate(("ID anggota", "Kode buku tersedia")):
            ttk.Label(form, text=label).grid(row=0, column=kolom, sticky="w")
        self.pilihan_anggota = ttk.Combobox(
            form, textvariable=self.anggota_dipilih, state="readonly", width=16)
```

```
130         self.pilihan_anggota.grid(row=1, column=0, padx=(0, 8))
131         self.pilihan_buku = ttk.Combobox(
132             form, textvariable=self.buku_dipilih, state="readonly", width=16)
133         self.pilihan_buku.grid(row=1, column=1, padx=(0, 8))
134         self._isian(form, "Tanggal transaksi (YYYY-MM-DD)", self.tanggal, 2, 20)
135         ttk.Button(form, text="Pinjam", command=self.pinjam).grid(
136             row=1, column=3, padx=4)
137         ttk.Button(form, text="Kembalikan pilihan", command=self.kembalikan).
    grid(
138             row=1, column=4)
139         ttk.Label(halaman, text="Untuk pengembalian, pilih satu baris riwayat "
140                   "dan isi tanggal transaksi.").pack(anchor="w", pady=(12, 0))
141         self.tabel_pinjam = self._tabel(halaman, [
142             ("id", "ID pinjam", 80), ("anggota", "Anggota", 90),
143             ("buku", "Buku", 80), ("pinjam", "Tanggal pinjam", 115),
144             ("jatuh", "Jatuh tempo", 115), ("kembali", "Tanggal kembali", 120),
145             ("status", "Status", 85),
146         ])
147
148     def _isi_tabel(self, tabel, baris):
149         dipilih = tabel.selection()
150         tabel.delete(*tabel.get_children())
151         for identitas, nilai in baris:
152             tabel.insert("", "end", iid=identitas, values=nilai)
153         if dipilih and tabel.exists(dipilih[0]):
154             tabel.selection_set(dipilih[0])
155
156     def segarkan(self):
157         buku = self.perpus.daftar_buku()
158         anggota = self.perpus.daftar_anggota()
159         self._isi_tabel(self.tabel_buku, [
160             (b.kode, (b.kode, b.judul, b.JENIS,
161                       "tersedia" if self.perpus.tersedia(b.kode) else "dipinjam
    "))
162             for b in buku
163         ])
164         self._isi_tabel(self.tabel_anggota, [
165             (a.id_anggota, (a.id_anggota, a.nama)) for a in anggota
166         ])
167         self._isi_tabel(self.tabel_pinjam, [
168             (p.id_pinjam, (p.id_pinjam, p.anggota.id_anggota, p.buku.kode,
169                            p.tanggal_pinjam.isoformat(),
170                            p.tanggal_jatuh_tempo.isoformat(),
171                            p.tanggal_kembali.isoformat() if p.tanggal_kembali
    else "-",
```

```
                           "aktif" if p.aktif else "selesai"))
            for p in self.perpus.daftar_peminjaman()
        ])
        id_anggota = [a.id_anggota for a in anggota]
        tersedia = [b.kode for b in buku if self.perpus.tersedia(b.kode)]
        self.pilihan_anggota["values"] = id_anggota
        self.pilihan_buku["values"] = tersedia
        if self.anggota_dipilih.get() not in id_anggota:
            self.anggota_dipilih.set("")
        if self.buku_dipilih.get() not in tersedia:
            self.buku_dipilih.set("")

    def _jalankan(self, aksi, pesan):
        try:
            aksi()
        except (PerpustakaanError, ValueError) as exc:
            messagebox.showerror("Operasi ditolak", str(exc), parent=self.root)
            return False
        self.berubah = True
        self.segarkan()
        self.status.set(f"{pesan} Perubahan belum disimpan.")
        return True

    def tambah_buku(self):
        def aksi():
            kelas = {"cetak": BukuCetak, "ebook": EBook}.get(self.jenis.get())
            if kelas is None:
                raise ValueError("Pilih jenis cetak atau ebook.")
            self.perpus.tambah_buku(kelas(self.kode.get(), self.judul.get()))
        if self._jalankan(aksi, "Buku terdaftar."):
            self.kode.set("")
            self.judul.set("")

    def tambah_anggota(self):
        aksi = lambda: self.perpus.tambah_anggota(
            Anggota(self.id_anggota.get(), self.nama.get()))
        if self._jalankan(aksi, "Anggota terdaftar."):
            self.id_anggota.set("")
            self.nama.set("")

    def pinjam(self):
        def aksi():
            if not self.anggota_dipilih.get() or not self.buku_dipilih.get():
                raise ValueError("Pilih anggota dan buku yang tersedia.")
            self.perpus.pinjamkan(self.anggota_dipilih.get(),
                                  self.buku_dipilih.get(),
```

```
                                    baca_tanggal(self.tanggal.get()))
        self._jalankan(aksi, "Peminjaman tercatat.")

    def kembalikan(self):
        def aksi():
            pilihan = self.tabel_pinjam.selection()
            if not pilihan:
                raise ValueError("Pilih satu baris peminjaman.")
            self.perpus.kembalikan(pilihan[0], baca_tanggal(self.tanggal.get()))

        self._jalankan(aksi, "Pengembalian tercatat.")

    def pilih_buku(self, event=None):
        pilihan = self.tabel_buku.selection()
        if pilihan:
            kode = pilihan[0]
            self.buku_dipilih.set(kode if self.perpus.tersedia(kode) else "")

    def simpan(self, event=None):
        try:
            self.perpus.simpan()
        except PerpustakaanError as exc:
            messagebox.showerror("Gagal menyimpan", str(exc), parent=self.root)
            return False
        self.berubah = False
        self.status.set("Data tersimpan.")
        return True

    def tutup(self):
        if self.berubah:
            jawaban = messagebox.askyesnocancel(
                "Simpan perubahan?", "Simpan perubahan sebelum keluar?",
                parent=self.root)
            if jawaban is None:
                return
            if jawaban and not self.simpan():
                return
        self.root.destroy()


def main():
    root = tk.Tk()
    root.withdraw()
    path = Path(__file__).resolve().parent / "data" / "perpustakaan.json"
    perpus = Perpustakaan(PenyimpananJSON(path))
    try:
```

```
263             perpus.muat()
264         except PerpustakaanError as exc:
265             messagebox.showerror("Data tidak dimuat", str(exc), parent=root)
266             root.destroy()
267             return 1
268         AplikasiPerpustakaan(root, perpus)
269         root.deiconify()
270         root.mainloop()
271         return 0
272
273
274 if __name__ == "__main__":
275     raise SystemExit(main())
```

Method yang diawali garis bawah dipakai untuk menyusun bagian tampilan atau menjalankan pekerjaan bantuan. Method publik seperti `pinjam()`, `kembalikan()`, dan `simpan()` menjadi callback. Fungsi lokal `aksi()` serta `lambda` menunda pemanggilan layanan sampai fungsi itu dijalankan oleh `_jalankan()`.

Fungsi `main()` membuat satu object `Tk`, menyembunyikannya sementara selama pemuatan, lalu menampilkan jendela setelah layanan siap. Jika pemuatan gagal, kotak pesan ditampilkan dengan parent yang jelas dan jendela dihancurkan. Jika berhasil, `mainloop()` menangani interaksi sampai pengguna menutup aplikasi.

Menjalankan dan Menelusuri Hasilnya

Jalankan dari folder proyek dengan perintah berikut. Hasil program berupa jendela, tabel, teks status, dan kotak pesan; tidak ada rangkaian `print()` seperti pada demo terminal.

Perintah menjalankan aplikasi

```
python gui.py
```

Untuk memperoleh hasil yang dapat dibandingkan, gunakan salinan proyek dengan folder data baru, lalu ikuti langkah berikut.

Mencoba Satu Siklus Peminjaman

1. Pada tab Buku, daftarkan B001 berjudul Dasar OOP dengan jenis cetak dan E001 berjudul Dasar Python dengan jenis ebook.
2. Pada tab Anggota, daftarkan M001 dengan nama Ani.
3. Pada tab Peminjaman, pilih M001 dan B001, isi tanggal `2026-10-27`, lalu klik Pinjam.
4. Periksa P001: tanggal jatuh tempo `2026-11-10`, tanggal kembali berupa tanda `-`, dan status aktif. B001 ditampilkan sebagai dipinjam dan tidak lagi muncul pada pilihan buku

tersedia.

5. Pilih baris P001, ubah tanggal transaksi menjadi `2026-11-02`, lalu klik Kembalikan pilihan. P001 menjadi selesai dan B001 tersedia kembali.
6. Pilih M001 dan E001, gunakan tanggal `2026-11-03`, lalu klik Pinjam. Transaksi P002 jatuh tempo pada `2026-11-10`, sesuai masa pinjam e-book tujuh hari.
7. Klik Simpan, tutup aplikasi, lalu buka kembali. Kedua buku, anggota, dan dua catatan riwayat harus tetap terlihat.

Tabel 13.5. Keadaan layar yang diharapkan pada percobaan.

Tahap	Buku	Riwayat
Pendaftaran selesai	B001 dan E001 tersedia	Belum ada transaksi
B001 dipinjam	B001 dipinjam; E001 tersedia	P001 aktif, jatuh tempo 10 November
B001 dikembalikan	B001 dan E001 tersedia	P001 selesai, kembali 2 November
E001 dipinjam	B001 tersedia; E001 dipinjam	P001 selesai dan P002 aktif
Simpan dan buka ulang	Sama dengan keadaan terakhir tersimpan	ID, tanggal, dan status tetap sama

Tanggal awal pada formulir diisi dengan tanggal komputer saat aplikasi dibuka. Pembaca harus menggantinya dengan tanggal pada langkah percobaan jika ingin mencocokkan hasil tersebut. ID P001 dan P002 berlaku untuk data baru; proyek yang sudah memiliki riwayat akan melanjutkan nomor yang tersimpan.

Memeriksa Penolakan dan Penutupan

Periksa pula masukan dan keputusan yang gagal. Keberhasilan GUI mencakup kemampuan menjelaskan penolakan sambil mempertahankan data yang benar. Gunakan tabel berikut sebagai daftar skenario yang dapat diulang pada salinan proyek.

Daftar pilihan buku tersedia merupakan bantuan tampilan, bukan pengganti pemeriksaan transaksi. Larangan peminjaman ganda sudah diperiksa pada layanan Bab 12 dan tetap berlaku ketika method dipanggil dari antarmuka lain. Pengujian GUI melengkapi pemeriksaan tersebut dengan menilai pilihan widget, perubahan tabel, pesan, dan keputusan penutupan.

JIKA JENDELA TERASA TIDAK MENANGGAPI

Handler berjalan pada event loop yang sama dengan tampilan. Pekerjaan yang lama dapat

Tabel 13.6. Pemeriksaan GUI di luar alur normal.

Percobaan	Hasil yang diharapkan
Daftarkan kode buku yang sama	Pesan penolakan; buku lama tetap ada dan isian masih dapat dikoreksi.
Pinjam tanpa memilih anggota atau buku	Pesan meminta pilihan; tidak ada transaksi baru.
Isi tanggal `2026-02-30`	Pesan format/tanggal tidak sah; riwayat tidak berubah.
Kembalikan tanpa memilih baris	Pesan meminta satu baris; tidak ada catatan yang berubah.
Kembalikan sebelum tanggal pinjam	Layanan menolak; transaksi tetap aktif.
Kembalikan catatan yang sudah selesai	Layanan menolak; tanggal kembali sebelumnya tetap ada.
Tutup setelah perubahan, lalu pilih Batal	Jendela tetap terbuka dengan data dan penanda perubahan yang sama.
Pilih Ya, tetapi penyimpanan gagal	Pesan kegagalan; jendela tetap terbuka dan perubahan belum dianggap tersimpan.
Buka salinan file JSON yang rusak	Pesan pemuatan gagal; jendela utama tidak melanjutkan dengan data kosong.

menunda respons jendela. Contoh ini membaca dan menulis file lokal berukuran kecil. Jika kelak ada unduhan atau pengolahan besar, pelajari pembagian pekerjaan dan penjadwalan pembaruan GUI; jangan menambahkan `sleep()` atau perulangan panjang ke handler.

13.5 Rangkuman

- GUI menerjemahkan pekerjaan pengguna menjadi formulir, pilihan, tombol, tabel, dan umpan balik yang jelas.
- `StringVar` menghubungkan teks dengan widget. Konversi format dilakukan sebelum nilai diteruskan ke layanan.
- Event handler didaftarkan sebagai rujukan method. `command` dan `bind()` mempunyai cara pemberian argumen yang berbeda.
- ID domain dipakai sebagai nilai pilihan dan identitas baris tabel, sehingga operasi tidak bergantung pada urutan baris atau keunikan nama.
- Tampilan disegarkan dari layanan setelah operasi berhasil; operasi yang ditolak tidak dianggap sebagai perubahan data baru.
- Model dan layanan menjaga aturan OOP, sedangkan GUI membaca input, memanggil operasi, dan menyajikan hasilnya.
- Penyimpanan yang gagal serta pembatalan penutupan tidak boleh dianggap sebagai keberhasilan menyimpan data.

13.6 Kuis Teoritis

Pilihan Ganda

1. Pendaftaran callback tombol yang benar adalah ...

   a. `command=self.pinjam()`
   b. `command=self.pinjam`
   c. `command="self.pinjam()"`
   d. `command=None` untuk menjalankan peminjaman

2. Keuntungan memakai ID peminjaman sebagai `iid` tabel adalah ...

   a. identitas catatan tetap dapat ditemukan walaupun urutan baris berubah
   b. semua tanggal otomatis menjadi sah
   c. layanan tidak lagi diperlukan
   d. catatan selesai otomatis dihapus

3. Pemeriksaan buku masih tersedia ketika transaksi dibuat menjadi tanggung jawab ...

a. warna tombol
b. teks judul jendela
c. layanan `Perpustakaan`
d. scrollbar tabel

4. Jika pengguna memilih Batal pada dialog penutupan, aplikasi harus ...

a. menutup tanpa menyimpan
b. menghapus perubahan di memori
c. menganggap penyimpanan berhasil
d. tetap membuka jendela dengan keadaan yang sama

5. Pernyataan yang tepat setelah pengembalian berhasil adalah ...

a. seluruh riwayat peminjaman buku dihapus
b. layanan berubah, kemudian GUI membaca keadaan terbaru untuk ditampilkan
c. GUI cukup mengubah tulisan status tanpa memanggil layanan
d. tanggal kembali pada model tetap kosong

KUNCI PILIHAN GANDA

1 b; 2 a; 3 c; 4 d; 5 b.

Uraian

1. Jelaskan perbedaan pemeriksaan format tanggal di GUI dan pemeriksaan tanggal kembali pada model.
2. Mengapa pilihan buku perlu diperbarui setelah peminjaman maupun pengembalian?
3. Telusuri apa yang terjadi jika pengguna memilih Ya saat menutup, tetapi penulisan file gagal. Sebutkan keadaan jendela dan penanda perubahan.

13.7 Praktik Mandiri

Praktik 1: Menyaring Riwayat Aktif

Tambahkan `ttk.Checkbutton` untuk menampilkan hanya peminjaman aktif. Gunakan parameter `aktif_saja` pada layanan. Hasil wajib: catatan selesai tersembunyi ketika filter aktif dan muncul kembali ketika filter dimatikan; tidak ada catatan domain yang dihapus.

Praktik 2: Mempermudah Pencarian Buku

Tambahkan isian pencarian judul pada tab Buku. Cocokkan teks tanpa membedakan huruf besar dan kecil, lalu tampilkan hasilnya pada tabel. Pertahankan kode buku sebagai `iid`. Hasil wajib: pencarian tidak mengubah koleksi layanan dan pemilihan baris hasil tetap mengarah ke buku yang benar.

Praktik 3: Memeriksa Keputusan Penutupan

Buat catatan pemeriksaan untuk kondisi tanpa perubahan, perubahan belum disimpan, Batal, Tidak, Ya berhasil, dan Ya dengan penyimpanan gagal. Gunakan salinan proyek atau penyimpanan percobaan. Hasil wajib: hanya penyimpanan berhasil yang menghapus penanda perubahan, dan kegagalan menyimpan tidak menutup jendela.

Nilai hasil praktik dari ketepatan perilaku, pemakaian method publik layanan, keterbacaan susunan widget, dan bukti pemeriksaan. Setelah alur tersebut dipahami, pembaca dapat memecah class GUI menjadi module tampilan per tab tanpa mengubah kontrak layanan.

REFERENSI TEKNIS LANJUTAN

Pelajari dokumentasi resmi Tkinter untuk hubungan widget, event loop, dan handler. Rincian `Notebook`, `Combobox`, `Treeview`, serta scrollbar dapat dibaca pada dokumentasi resmi `tkinter.ttk`. Gunakan dokumentasi yang sesuai dengan versi Python yang dipakai.

Bab 14

Penutup

Merangkum perjalanan dari class pertama hingga aplikasi utuh.

14

DI BAB INI

- Rangkuman materi
- Langkah selanjutnya

14.1 Rangkuman Materi

Kita memulai dengan class, object, attribute, method, dan constructor untuk menyatukan data serta perilaku yang berkaitan. Encapsulation dan property menjaga perubahan data agar mengikuti aturan. Class method, static method, dan decorator memperluas cara menyusun perilaku pada class.

Inheritance, abstraksi, dan polymorphism membantu membentuk kontrak serta variasi perilaku. Association, aggregation, dan composition menjelaskan hubungan antar-object. Pemilihannya mengikuti kebutuhan serta tanggung jawab class.

Module dan package mengatur kode, exception handling menangani kegagalan, dan file handling mempertahankan data setelah program ditutup. Melalui GUI, input pengguna dihubungkan dengan operasi aplikasi menggunakan widget dan event handler.

Proyek perpustakaan mempertemukan materi tersebut: menganalisis kebutuhan, merancang class dan relasi, mengimplementasikan aturan, lalu membuat antarmuka. Kita belajar menjaga data, menjelaskan penolakan, serta memeriksa hasil setelah aplikasi disimpan dan dibuka kembali.

14.2 Langkah Selanjutnya

Kemampuan merancang program akan berkembang melalui latihan dan perbaikan. Pilih satu pengembangan kecil dari proyek, misalnya pencarian buku atau laporan peminjaman terlambat. Tuliskan hasil yang diharapkan, tentukan class yang bertanggung jawab, lalu implementasikan dan periksa skenario berhasil maupun gagal.

Cobalah pula menerapkan langkah yang sama pada masalah lain, seperti peminjaman alat laboratorium atau pengelolaan stok sederhana. Gunakan dokumentasi Python ketika menemui perilaku yang belum dipahami. Saat terjadi kesalahan, baca pesannya, telusuri perubahan data, dan perbaiki penyebabnya satu per satu.

TERUS MENCOBA DAN BERLATIH

Terima kasih telah belajar bersama melalui buku ini. Luangkan waktu untuk menulis kode sendiri, mengubah contoh, dan memeriksa hasilnya. Setiap latihan memberi kesempatan untuk memahami keputusan rancangan dengan lebih baik. Selamat mencoba dan terus berlatih agar semakin mahir menggunakan OOP.

Daftar Pustaka

Sumber yang dirujuk dalam bab ini.

[Edcorner Learning, tt] Edcorner Learning (t.t.). *Python Object Oriented Programming Exercises: Become a Pro Developer*. Penerbit tidak tercantum. Tahun terbit tidak tercantum pada salinan digital yang digunakan.

[Lutz, 2025] Lutz, M. (2025). *Learning Python: Powerful Object-Oriented Programming*. O'Reilly Media, Sebastopol, CA, 6 edition.

[North, 2025] North, M. (2025). *Object-Oriented Programming in Python: Design and Build Programs Using Classes, Objects, and the Four Pillars of OOP*, volume 1 of *Think, Code, Create – Advanced Python Programming Series*. Northward House Publishing, Brisbane, Queensland, Australia.

[Wilson, 2024] Wilson, G. (2024). *Software Design by Example: A Tool-Based Introduction with Python*. CRC Press.